

창고에서 반찬통까지

품질검증(P5) · 스테이징(P6-1) · 마트(P6-2) — BigQuery 를 처음 보는 사람을 위한 안내서

작성 2026-08-18 · docs/ 위키(2층) 소속 · 원본은 docs/stg-mart-guide.html

읽는 사람 BigQuery 를 써 본 적 없고 "스테이징"이 무엇인지 모르는 팀원

수치 추정치 아니라 실제로 질의해서 얻은 값이다. 확인 시점 2026-08-18

범위 데이터가 BigQuery 에 들어온 뒤부터 대시보드 직전까지. 그 앞은 pipeline-overview.pdf

- 0 다섯 단어만 먼저 — 데이터셋 · 표 · 뷰 · 스캔량 · 파티션
- 1 데이터는 어떻게 들어오나 (P4 적재)
- 2 P5 품질검증 — 온 것을 믿어도 되나
스테이징이 무엇인가
- 3 인가 — 이 문서의 핵심
- 4 SQL 을 왜 그렇게 썼나 — 실제 코드 넷
- 5 마트가 무엇인가 — 주간 규칙 넷
- 6 무슨 결정을 했고 왜 그렇게 했나
- 7 맞는지 어떻게 확인했나
- 8 만들다가 드러난 것 둘
- 9 앞으로 — Airflow 계획
- 10 돈과 한도
- 11 용어집 · 명령 요약

한 장 요약

데이터를 세 번 옮겨 담는다. 같은 데이터를 세 번 쓰는 게 아니라, 목적이 다른 세 층을 만든 것이다.

① **raw** — 원본 그대로

40개 표 · 1억 2,373만 행 · 8.74 GiB · 제약도 정리도 없다 · 파이프라인만 읽는다

↓ P5 품질검증 — 온 것을 믿어도 되나 (430항목)

② **stg** — 실수를 구조적으로 막은 층 뷰 10개 · 저장 0

사람이 매번 기억해서 걸어야 했던 조건을 미리 걸어 둔 층. 분석은 여기서 시작한다

↓ P6-2 지표를 미리 계산

③ **mart** — 대시보드가 읽는 층 표 8개 · 495 MiB

1억 2천만 행을 미리 접어 둔 결과. 화면은 여기만 본다

↓ P7 루커 스튜디오 (아직 안 붙였다)

왜 굳이 세 층인가 — 한 문장

raw 를 직접 보면 조용히 틀리고, 매번 다시 계산하면 비싸기 때문이다. **stg** 는 "틀리지 않게" 하는 층이고, **mart** 는 "빠르고 싸게" 하는 층이다. 둘은 서로 다른 문제를 푼다.

0. 다섯 단어만 먼저

BigQuery 를 안 써 봤다면 이 다섯 개만 알면 이 문서를 읽을 수 있다.

말	쉬운 비유	정확히는
데이터셋	폴더	표를 담는 칸. 우리는 <code>raw</code> · <code>stg</code> · <code>mart</code> 셋을 쓴다
표(테이블)	엑셀 시트	데이터가 실제로 저장 돼 있다. 만들 때 계산이 끝나고, 이후에는 읽기만 한다
뷰(view)	저장해 둔 질문	데이터를 갖고 있지 않다. "이렇게 꺼내라"는 문장만 저장된 것이라, 읽을 때마다 원본에서 다시 꺼낸다. 그래서 저장 용량 이 0이고 항상 최신이다
스캔량	수도 계량기	BigQuery 는 읽은 데이터 양으로 돈을 받는다. 표를 많이 만들어 두는 것보다 큰 표를 자주 읽는 것 이 비싸다. 월 1 TiB 까지 무료
파티션 · 클러스터	서랍 칸막이 · 칸 안 정렬	날짜로 서랍을 나눠 두면(<code>PARTITION BY</code>) "6월 첫 주"를 볼 때 6월 서랍만 연다. 스캔량이 줄어 그대로 돈이 준다

여기서 오는 오해 하나

"표를 많이 만들면 돈이 많이 나오지 않나?" — **아니다.** 저장은 10 GiB 까지 무료이고 우리 마트 8개는 합쳐 **495 MiB**다. 반대로 **대시보드**가 `raw` 를 직접 보면 화면을 새로 그릴 때마다 1억 2천만 행을 읽는다. **마트를 만드는 이유가 바로 돈이다.**

`--dry-run` — 켜기 전에 계량기부터 본다

이 프로젝트의 모든 BigQuery 명령에는 `--dry-run` (또는 `--estimate`)이 있다. **실행하지 않고 "이 쿼리는 몇 GiB 를 읽을 것이다"만 알려준다.** 공짜다. 습관으로 만들어 두면 실수로 큰돈을 쓰지 않는다.

```
# 문법과 스캔량만 본다 - 아무것도 만들지 않는다
python bigquery/build.py --layer stg --dry-run
```

1. 데이터는 어떻게 들어오나 (P4 적재)

세 층 이야기를 하기 전에, ①번 층이 어떻게 채워지는지를 먼저 알아야 한다. 여기서 생긴 성질이 stg 가 존재하는 이유가 되기 때문이다.

1.1 두 곳에서 와서 한 표에 담긴다

원천	_source	무엇
Supabase	supabase	실유저 — 지인 23명이 실제로 쓴 흔적
로컬 Postgres	local	합성 데이터 — 지어낸 20,000명 · 12개월

둘은 같은 표로 들어간다. 그런데 양쪽 다 id 가 1번부터 시작한다 — 즉 실유저 3번과 합성 3번이 한 표 안에 같이 있다. 그래서 적재할 때 _source 컬럼을 붙이고 키를 (_source, id) 쌍으로 쓴다.

이 설계가 만드는 함정 — stg 가 태어난 이유

분석할 때 JOIN ... USING(id) 처럼 _source 를 빠뜨리면 실유저의 투표에 합성 유저의 프로필이 붙는다. 오류는 나지 않는다. 숫자만 커진다.

실측 — vote_item × app_user 를 두 방식으로 세어 봤다: _source 없이 13,459,564행, 넣고 13,445,278행. 14,286행이 소리 없이 부풀었다.

1.2 증분 적재 — 바뀐 것만 가져온다

매번 1억 행을 다시 올릴 수는 없다. 그래서 표마다 워터마크(마지막으로 가져온 시각)를 기록해 두고, 그보다 새로 바뀐 행만 가져온다. 기준 컬럼은 updated_at 하나로 통일했다.

성질	왜 그렇게 했나
지운 행도 남긴다	원천에서 삭제돼도 BigQuery 에서는 지우지 않고 _deleted_at 에 시각만 찍는다. raw 는 이력을 잃지 않는다는 원칙. 대신 분석할 때 _deleted_at IS NULL 이 거의 항상 필요해진다 — 이것도 stg 가 대신 걸어 준다
적재 자체는 무료다	배치 로드 잡을 쓴다. 스트리밍이었으면 GB 당 과금이었을 것이다
파티션 순서로 꺼낸다	파티션 표는 하루 4,000회의 파티션 수정만 받는데, 적재 한 덩어리가 수십 일치에 흩어지면 그 한도를 금방 먹는다. 실제로 vote_candidate 2,620만 행에서 막혔다. 그래서 날짜 순서로 정렬해 꺼낸다

증분은 조용히 틀린다 — 실제로 12일 멈춰 있었다

2026-08-18 에 확인하니 BigQuery 의 워터마크가 8월 6일에 멈춰 있었다. Supabase 에는 8월 13일까지의 투표가 있었는데도 아무 오류도 나지 않았다. 쿼리는 잘 돌았고 답도 나왔다 — 그냥 12일치가 없는 답이었다. 그래서 적재 뒤에 행 수를 세어 대조하고(verify_load.py), 품질검사가 워터마크가 언제 것인지를 함께 찍는다.

2. P5 품질검증 — 온 것을 믿어도 되나

이름이 비슷한 두 검사가 있는데 묻는 것이 다르다. 이 구분이 P5 의 전부다.

무엇	묻는 것
<code>verify_load.py</code>	원천의 행이 다 왔나 (개수를 센다)
<code>qa/quality_check.py</code>	온 행을 믿어도 되나 (내용을 본다)

2.1 왜 검사가 여기 있나

`raw` 층은 일부러 제약이 없다. 적재기가 모든 컬럼을 NULL 허용으로 만들고 외래키도 걸지 않는다. 원천 스키마가 조금 달라져도 적재가 통째로 죽지 않게 하려는 선택이다. 그 대신 검증을 적재 시점이 아니라 이 단계에 몰아 두었다.

2.2 검사 430개는 손으로 적지 않았다

살아 있는 스키마(`docs/erd.json`)를 읽어 표마다 자동으로 만든다. 표가 늘면 검사도 저절로 는다 — 사람이 검사 추가를 잊어서 생기는 구멍이 없다.

구분	개수	보는 것
유일성	40	<code>(_source, id)</code> 가 중복되지 않는가
필수값	276	원천이 NOT NULL 인 컬럼에 NULL 이 없는가
참조	75	부모 없는 자식 행이 없는가 (같은 <code>_source</code> 안에서만)
시각	39	미래 시각 · 뒤집힌 순서(<code>voted_at < served_at</code> 등)
적재 상태	—	워터마크가 언제 것인가 (정보)
합계	430	두 원천 다 통과 (2026-08-18)

한 번 도는 데 11.7 GiB 를 읽는다. 월 무료 한도의 1.1% 라 매일 돌려도 한 달 34% 다.

2.3 언제 다시 도나

- 적재를 돌린 뒤
- 스키마를 바꾼 뒤 — 컬럼을 추가하면 `updated_at` 이 안 움직여 증분이 못 잡고, 그 컬럼이 통째로 NULL 로 남는다. "필수값" 검사가 그것을 잡는다

3. 스테이징이 무엇인가

한 문장

스테이징(`stg`)은 "raw 를 그대로 쓰지 못하게 만드는 층"이다. 분석할 때마다 사람이 기억해서 걸어야 했던 조건들을 미리 걸어 둔 것이고, 기억에 기대던 규칙을 구조로 바꾼 것이다.

3.1 주방으로 비유하면

층	주방	뜻
<code>raw</code>	창고	배달 온 그대로. 흠 묻은 채소, 안 쓸 것도 섞여 있다. 버리는 것 없이 다 쌓아 둔다
<code>stg</code>	손질대	씻고 다듬어 놓은 상태. 요리는 아직 아니다. 무엇을 만들든 여기서 시작한다
<code>mart</code>	반찬통	이미 만들어 담아 둔 것. 꺼내면 바로 먹는다. 대신 미리 정한 반찬만 있다

손질대가 없으면 요리할 때마다 창고에서 흠 묻은 채로 꺼내 씻어야 한다. 바쁠 때 한 번 안 씻으면 그대로 상에 올라간다 — 그게 지금까지의 문제였다.

3.2 `stg` 가 막는 실수 여섯 가지

전부 실제로 물렸던 것들이다. 공통점은 오류가 나지 않고 숫자만 틀린다는 것.

막는 실수	어떻게 막나
두 원천의 id 가 겹친다	대리키 <code>user_key = 'local-123'</code> 를 만든다. 원천이 다르면 키가 달라 <code>USING(id)</code> 로 잇는 것이 불가능해진다
지운 행의 유령	<code>_deleted_at IS NULL</code> 을 뷰 안에서 한 번만 건다
시간대가 9시간 밀린다	저장은 UTC 다. <code>signup_date</code> 처럼 한국 날짜를 미리 계산해 심는다. 안 바꾸면 밤 10시가 낮 1시로 보인다
광고로 연 무료 힌트가 '구매'로 세어진다	<code>is_paid_hint · is_ad_hint</code> 로 갈라 둔다. 하트를 낸 것과 광고를 본 것은 다른 사건이다
스텝 충전이 매출로 세어진다	MVP 의 충전 버튼은 돈이 안 오간다. <code>is_revenue</code> 하나로 못 박는다
제재당한 유저를 통째로 빼면 과거까지 사라진다	유저가 아니라 시각으로 자른다 — <code>valid_until</code> . 합성 데이터에는 제재 뒤에도 투표하는 유저가 34명 있다

3.3 왜 표가 아니라 뷰인가

`stg` 는 전부 뷰다(저장 0). 이유는 둘이다.

- 저장 여유가 없다. `raw` 가 이미 8.74 GiB 라 무료 10 GiB 중 남은 게 1 GiB 남짓이다. `stg` 를 표로 구우면 그 여유가 사라진다.
- 읽는 쪽이 적다. `stg` 를 읽는 것은 `mart` 와 분석자뿐이다. 하루에 수십 번 읽히는 대시보드와 달라서, 읽을 때마다 다시 계산해도 비용이 작다.

덤으로 **항상 최신**이다. raw 에 새 행이 들어오면 stg 는 **다시 만들 필요가 없다** — 뷰는 데이터를 갖고 있지 않기 때문이다. 반대로 **mart** 는 반드시 다시 구워야 한다.

3.4 지금 있는 뷰 10개

뷰	한 행이 뜻하는 것
stg_user	유저 한 명 (+ 소속 · 지역 · 탈퇴 · 제재 · valid_until)
stg_user_session	앱 접속 한 번 (+ 체류시간 · 중복 로그 표시)
stg_vote_session	투표 한 판 (+ 완료 여부)
stg_vote_item	출제된 질문 한 개
stg_vote_received	받은 투표 한 건 (+ 열람 · 공개 단계)
stg_hint_purchase	힌트를 연 기록 (+ 유료/광고 구분)
stg_heart_transaction	하트 원장 한 줄 (+ 적립/소비 분리)
stg_heart_purchase	충전 시도 (+ 스텝/매출 구분)
stg_friend_edge	한 사람이 본 친구 관계 (관계 1건 = 두 행)
stg_report	신고 한 건 (+ 사유 이름 · 처리 여부)

뒤의 넷과 stg_user 의 지역은 **대시보드 때문에 늘었다**. 스케치 5쪽 중 2·3·5쪽이 기존 6개로는 그려지지 않았다 — 받은 투표 · 신고 · 친구 관계가 아예 없었기 때문이다.

4. SQL 을 왜 그렇게 썼나 — 실제 코드 넷

파일은 `bigquery/staging/*.sql` 에 있다. 파일 이름의 번호가 실행 순서다. 여기서는 "왜 이렇게 생겼는지"만 본다.

4.1 대리키 — 실수를 불가능하게 만든다

```
CONCAT(u._source, '-', CAST(u.id AS STRING)) AS user_key
-- 'local-123' · 'supabase-3'
```

id 를 그대로 두면 사람이 `_source` 를 기억해야 한다. 문자열로 붙여 버리면 합성의 3번과 실유저의 3번이 다른 값이 되어, 잘못 이으면 결과가 0행이 된다. 틀린 답 대신 빈 답이 나오는 것이 이 설계의 목적이다 — 빈 답은 사람이 알아챈다.

4.2 친구 관계를 두 행으로 편다

원본은 관계 하나를 한 줄로 적되, 두 사람을 id 순서로 정렬해 넣는다 (`user_low_id < user_high_id`). 저장에는 좋지만 분석에는 함정이다.

```
-- 낮은 쪽에서 본 관계
SELECT user_low_id AS user_id, user_high_id AS friend_id ... FROM friendship
UNION ALL
-- 높은 쪽에서 본 같은 관계
SELECT user_high_id AS user_id, user_low_id AS friend_id ... FROM friendship
```

안 폼으면 어떻게 됐나

`user_low_id` 만 세면 친구 수가 정확히 절반이 되고, id 가 작은 사람에게만 친구가 있는 것처럼 보인다. 역시 오류는 안 난다.

그래서 뷰 이름도 `friendship` 이 아니라 `friend_edge` 다 — 한 행은 "관계"가 아니라 "한 사람이 본 관계"라는 뜻. 확인도 쉽다: $795,656 = 397,828 \times 2$ 여야 한다.

4.3 이름을 바꿔 실은 것 — `relation_source`

친구 표에는 이미 `source` 라는 컬럼이 있다(어떻게 맺어졌나 — 초대코드·검색·추천). 그런데 우리는 적재 원천 `_source` 를 `source` 로 실는 관례를 쓴다. 이름이 부딪히면 나중에 반드시 잘못 읽는다. 그래서 관계 출처를 `relation_source` 로 바꿔 실었다.

4.4 일부러 계산하지 않은 것 — 신고 에이징

`stg_report` 는 "며칠째 밀렸나"를 계산하지 않는다. 뷰이기 때문이다. 뷰는 읽을 때마다 다시 계산되므로 "7 일 이상 미해결 8,898건"이 조회하는 시각마다 달라진다. 화면에는 그 사실이 드러나지 않는다.

그래서 자르는 시각은 마트가 정하고, `as_of_date` 컬럼으로 언제 기준인지를 표에 박아 둔다. `stg` 는 `created_at` 과 `reviewed_at` 이라는 사실만 실는다.

여기서 보이는 원칙

stg 는 "사실"을 다듬고, mart 는 "판단"을 굽는다. 며칠 밀렸는지는 사실이 아니라 기준 시각을 정해야 나오는 판단이다.

5. 마트가 무엇인가

한 문장

마트(mart)는 "대시보드가 읽을 답을 미리 계산해 둔 층"이다. stg 가 틀리지 않게 하는 층이라면, mart 는 빠르고 싸게 하는 층이다.

5.1 왜 여기는 뷰가 아니라 표인가

루커 스튜디오 같은 대시보드는 화면을 다시 그릴 때마다 쿼리를 날린다. 필터를 한 번 바꾸면 또 날아간다. 뷰로 두면 그때마다 1억 2천만 행을 다시 읽는다. 표로 구워 두면 495 MiB 만 읽는다. stg 와 정반대의 선택 이고, 이유도 정반대다.

5.2 그레인 — "한 행이 무엇인가"

마트를 만들 때 제일 먼저 정하는 것이 그레인이다. "이 표의 한 행은 무엇 하나를 뜻하는가". 유저 한 명인 가, 하루인가, 유저의 하루인가. 그레인이 정해지면 그 표로 답할 수 있는 질문의 범위가 정해진다.

5.3 ⚠️ 주간 필터가 설계를 바꿨다 — 규칙 넷

이 프로젝트에서 마트 설계를 실제로 결정한 것은 지표 목록이 아니라 "주 단위로 좁혀 전주와 비교하고 싶다"는 한 줄이었다.

규칙	여기면 무슨 일이 생기나
① 모든 표에 날짜 컬럼을 둔다	루커의 '전주와 비교' 기능은 날짜 축이 있는 데이터에서만 켜진다. 없으면 증감 표시가 아예 생기지 않는다
② 비율을 굵지 않는다. 분자·분모를 심는다	"처리율 45.4%"를 미리 저장하면, 주간 값이 일별 비율의 평균이 된다. 1건 중 1건(100%)과 100건 중 50건(50%)의 평균은 75% 지만 실제로는 50.5% 다
③ 사람 수를 미리 세지 않는다	같은 사람이 월·수·금 접속하면 일별로는 1+1+1 이지만 주간으로는 1 명이다. 미리 세어 두면 기간을 바꿀 때 다시 셀 수 없다
④ 퍼널·분포는 사건이 일어난 날로 자른다	축이 잘못 잡히면 기간을 좁힐 때 같은 대상이 두 주에 중복으로 잡힌다

규칙 ③ 때문에 표 하나가 통째로 바뀌었다

원래 계획은 유저×월 표였다. 그러면 주간 활성 유저를 낼 수 없다. 그래서 유저×활동일 — "이 사람이 이 날 무엇을 했나"를 한 행으로 하는 표로 바꿨다. 원자 행이 사람 단위로 남아 있으면 루커가 어떤 기간에서든 다시 셀 수 있다. 덤으로 일·주·월 지표가 표 하나에서 전부 나온다.

5.4 표 여덟 개

표	한 행 = 무엇	행 수	어디에 쓰나
---	----------	-----	--------

<code>mart_user</code>	유저 한 명	20,018	가입·해금·첫 투표·탈퇴, 퍼널 플래그, 친구 수 구간
<code>mart_user_activity</code>	유저의 하루	2,288,001	사람 수는 전부 여기서 센다 — MAU·WAU·주간 투표자
<code>mart_daily</code>	하루 × 소속	33,786	더할 수 있는 것만 — 가입 수·투표 수·하트·매출
<code>mart_funnel_step</code>	퍼널의 한 단계	272,890	가입→...→이름 공개 · 수신→열람→힌트
<code>mart_distribution</code>	분포의 한 구간	396,197	문항 도달 · 열람→첫 힌트 지연 · 힌트 종류
<code>mart_report</code>	신고 한 건	31,026	상태·유형·에이징 (작아서 접지 않고 그대로 굽는다)
<code>mart_heart_flow</code>	하루 × 하트 유형	197,879	하트가 어디서 들어와 어디로 나가나
<code>mart_backlog_weekly</code>	주 스냅샷	3,435	그 주 말 기준으로 신고가 몇 건 밀려 있었나
합계		약 325만	495 MiB (저장 무료 한도의 5%)

5.5 마지막 표만 성격이 다르다

적체는 사건이 아니라 상태다. "이번 주에 신고가 몇 건 들어왔나"는 접수일로 셀 수 있지만, "이번 주 말 기준으로 몇 건 밀려 있나"는 그 시점의 상태라 사건일 축으로는 안 나온다. 그래서 주마다 스냅샷을 만든다.

```
-- 그 시점에 "접수됐고 아직 안 본" 것을 계산으로 복원한다
created_at <= T AND (reviewed_at IS NULL OR reviewed_at > T)
```

과거를 계산으로 되살릴 수 있으므로 매번 통째로 다시 구워도 이력이 사라지지 않는다. 쌓아 두는 표가 아니라서 다른 표와 갱신 방법이 같다.

6. 무슨 결정을 했고 왜 그렇게 했나

결정	왜	기각한 대안
stg 는 뷰, mart 는 표	저장 여유가 1 GiB 뿐 · stg 는 읽는 쪽이 적다. 대시보드는 반대로 자주 읽는다	stg 도 표로 굽기 — 남은 여유가 사라진다
유저×활동일 팩트를 둔다	사람 수는 기간이 바뀌면 다시 세야 한다. 원자 행이 사람 단위여야 가능하다	유저×월 표 — 주간 MAU 를 못 낸다
비율을 굽지 않는다	미리 계산한 비율은 기간을 합칠 때 '평균의 평균'이 된다	처리율·전환율을 컬럼으로 저장 — 편하지만 틀린다
친구 관계를 두 행으로 편다	한쪽만 세면 친구 수가 절반이 되고 오류는 안 난다	마트에서 매번 UNION — 실수를 막는 자리가 뒤로 밀릴 뿐
"누가 나를 뽑았나" (voter_id)를 실지 않는다	앱에서 하트를 받고 파는 유료 정보다. 대시보드가 쓰지도 않는다	일단 실어두기 — 필요해질 때 여는 편이 안전하다
탈퇴자를 빼지 않고 표시만	빼면 이탈률의 분자가 사라진다. 거를지는 보는 쪽이 고른다	차원 표에서 제외 — 되돌릴 수 없다
제재는 유저가 아니라 시각으로 자른다	유저를 통째로 빼면 제재 전의 정상 활동까지 사라진다	제재 유저 제외 — 34명의 과거가 통째로 날아간다
신고 에이징은 mart 가 계산한다	뷰로 두면 조회 시각마다 답이 달라지고 그 사실이 안 보인다	stg 에서 CURRENT_DATE 사용
유저 퍼널은 독립 플래그로 둔다	힌트는 받은 투표에서 사는 것이라 투표 세션과 인과가 없다. 누적 조건부로 강제하면 세션을 안 끝내고 힌트만 산 사람이 사라진다	단조 감소하게 강제 — 대신 화면을 막대 묶음으로 그리기로 했다
이름 공개는 hint_type 으로 센다	합성 데이터의 reveal_status 가 갱신되지 않는다(8장)	생성기 수정 — 다음 판본에서 다른 어긋남과 함께

7. 맞는지 어떻게 확인했나

7.1 이미 아는 답과 맞춰 본다

노트북이 **raw** 를 직접 읽어 낸 값이 있다. 같은 값이 stg-mart 를 거쳐도 나오면 층이 제대로 선 것이다.

지표	노트북(raw 직접)	마트
신규 가입	20,000	19,995
접속 사용자	18,714	18,706
투표 참여	16,562	16,552
탈퇴	1,999	1,999
총매출	₩47.38M	₩47.22M
미해결 신고	10,140	10,123

왜 똑같지 않은가 — 일부러 어긋난다

노트북은 제재를 아무 데서도 거르지 않는다. 마트는 `valid_until` 로 제재 이후 활동을 잘라낸다. 그래서 마트가 조금 작게 나오는 것이 정상이다. 크게 나오면 그때가 버그다. 이렇게 "어긋나야 하는 방향"을 미리 적어 두면 다음 사람이 정상을 버그로 오해하지 않는다.

7.2 주간이 진짜 되는지 — 이번 작업의 핵심 확인

같은 값을 두 경로로 구해 맞춰 봤다(2026-06-01~07 한 주).

확인	결과
주간 합계	마트 386,252 = stg 직접 386,252 일치
주간 활성 유저	마트 1,384 = stg 직접 1,384 일치
전주 대비	1,533 → 1,384(-149) → 1,303(-81) 로 계산된다
적체 스냅샷	주마다 값이 달라진다 (9,508 → 10,123)

여기서 사람 수를 미리 세어 둔 표가 하나라도 있었으면 값이 어긋난다. 이 검증의 목적이 그것이다.

8. 만들다가 드러난 것 둘

① 합성 데이터가 앱과 다르게 동작한다

받은 투표에는 "얼마나 공개됐나"를 적는 `reveal_status` 컬럼이 있다. 실측해 보나:

	<code>reveal_status</code> = 공개됨	이름 공개 힌트 구매
합성 데이터	0건	8,411건 · 4,921명
실유저	10건	10건 (일치)

생성기가 이름을 공개를 팔면서 이 컬럼을 갱신하지 않는다. 앱은 갱신한다 — 실유저 쪽이 10 대 10 으로 정확히 맞는 것이 증거다. 두 원천을 함께 보지 않았으면 못 찾았을 종류다.

이 컬럼으로 퍼널을 켜다 보면 마지막 단계가 0 이 되고, 차트는 멀쩡히 그려지며, "아무도 이름을 안 산 다"는 틀린 결론만 남았을 것이다.

② 대시보드 스케치의 숫자 하나가 재현되지 않는다

스케치에는 "이름 공개 구매 6,701명"이 적혀 있다. 그런데 이름 공개 힌트를 산 사람은 통틀어 4,921 명이다. 불가능한 숫자다 — 스케치 쪽이 틀렸다.

루커에 실제로 붙일 때 마트 값으로 고친다. 그림이 먼저 나오면 그림의 숫자를 믿게 되는데, 층을 세워 두면 이렇게 대조가 된다.

9. 앞으로 — Airflow 계획

9.1 지금

Airflow(자동 실행기)는 이미 돌고 있지만 맨 앞 두 걸음만 한다.

```
ping_raw_load    매일 새벽 4시 (한국 시간)
├─ extract_load  Supabase → BigQuery raw 증분 적재
└─ verify       행 수 대조
```

나머지 — 품질검증과 마트 굶기 — 는 사람이 손으로 돌린다. 그래서 8월 6일부터 18일까지 12일 동안 아무도 모르게 멈춰 있었다.

9.2 다음 — 네 걸음으로 늘린다

데이터가 갱신될 때 실제로 필요한 것은 네 줄이고, 이것이 그대로 DAG 가 된다.

```
extract_load → verify → quality_check → build --layer mart
```

걸음	왜 필요한가
① 적재	새 행을 가져온다. 여기가 유일한 입구다

② 대조	증분은 조용히 틀린다. 매번 센다
③ 품질	온 행을 믿어도 되는지 본다 (430항목)
④ 마트 굶기	안 하면 오류 없이 옛날 숫자가 뜬다. 표는 만든 시점에 굳은 값이기 때문이다. stg 는 뷰라 안 해도 된다

④ 는 ① 과 성격이 다르다

적재(①)는 행을 밖에서 안으로 나른다. 마트 굶기(④)는 BigQuery 안에서 계산만 한다 — 오가는 것은 SQL 글자 몇 KB 뿐이다. 그래서 느리지도, 위험하지도 않다. 갈아끼우는 동안에도 옛 표가 보이므로 굶는 중에 대시보드를 열어도 반쯤 만들어진 숫자가 보이지 않는다.

9.3 그다음 — 주간 배달

대시보드는 사람이 열어야 보인다. 안 열면 안 본다. 그래서 주 1회 표지 요약 + 대시보드 PDF 를 보내는 것까지가 계획이다. PDF 발송은 루커 스튜디오의 내장 예약 발송을 쓴다 — DAG 로 만들면 헤드리스 브라우저를 떠안게 되고 얻는 것이 없다.

지킬 것	왜
실패도 배달한다	적재가 멈추면 메일이 안 오는 게 아니라 "이번 주 데이터 없음 — 적재 실패"가 와야 한다. 안 오는 것과 문제없는 것을 구분할 수 없으면 12일 멈춰도 아무도 모른다
합성이 안 변해도 보낸다	"변화 없음"을 명시해 보낸다. 안 보내면 파이프라인이 죽은 것과 구분되지 않는다
실유저에는 비율을 쓰지 않는다	23명 · 받은 투표 616건에서 "열람률 70.5%"는 두세 건만 움직여도 몇 %p 씩 튜다. 절대 수와 "처음 있는 일"(첫 결제·첫 탈퇴·첫 신고)로 보낸다
원천을 파라미터로 받는다	실유저는 매일, 합성은 재생성했을 때만 돈다. 같은 코드로 둘 다 되게 한다

⚠️ Airflow 스케줄은 UTC 다

월요일 아침 9시(한국)에 받으려면 일요일 00:00 UTC 로 걸어야 한다. 이 프로젝트에서 시간대는 이미 두 번 문제를 냈다(저장 UTC · 집계 KST).

10. 돈과 한도

항목	무료 한도	지금
저장	10 GiB	raw 8.74 GiB + stg 0 + mart 0.48 GiB $\approx$ 92%
쿼리(읽은 양)	월 1 TiB	품질검사 11.7 GiB/회 · 마트 굽기 7.3 GiB/회 — 매일 돌려도 여유
일일 하드캡	직접 건 상한	실수로 큰돈이 나가지 않게 스스로 채운 자물쇠

실제로 걸렸다 — 2026-08-18

마트를 처음 구우려는데 Custom quota exceeded 로 막혔다. 그날 이미 **136개 잡 · 30.5 GiB** 를 쓴 뒤였다(품질검사 등). **이건 사고가 아니라 장치가 작동한 것이다** — 이 프로젝트는 BigQuery 샌드박스가 아니라서 한도를 넘으면 **막히지 않고 과금된다**. 하드캡이 그 "막힘"을 되살린다.

해결은 둘 중 하나 — **콘솔에서 캡을 올리거나**, 태평양 자정(한국시간 오후 4~5시)의 **자동 리셋**을 기다리거나. 이번에는 캡을 올렸다.

비용이 커지는 곳은 표 개수가 아니다

쿼리 횟수가 지배한다. 대시보드가 raw 를 직접 본다면 (차트 20개 × 하루 30회 × 30일) 월 17 TiB 를 넘겨 수십만 원이 된다. 마트를 만드는 이유가 여기 있다 — **같은 화면을 495 MiB 로 그린다**.

11. 용어집

말	뜻
raw / stg / mart	원본 / 손질한 것 / 미리 만들어 둔 것
뷰	저장된 질문. 데이터를 갖고 있지 않아 용량 0, 항상 최신
증분 적재	전부가 아니라 바뀐 것만 가져오기
워터마크	"여기까지 가져왔다"는 표시. 이게 멈추면 조용히 낡는다
대리키	local-123 처럼 원천을 붙여 만든 키. 잘못된 조인을 불가능하게 한다
그레인	"한 행이 무엇 하나를 뜻하는가"
코호트	같은 시점에 시작한 집단. 여기서는 "그 주에 가입한 사람들"
퍼널	단계별로 얼마나 남는가. 단, 우리 유저 퍼널은 단조 감소하지 않는다 (6장)
distinct	중복을 뺀 개수. 사람 수 가 대표적이라 미리 세면 안 된다
스텝	진짜처럼 생겼지만 실제로는 동작하지 않는 것. 우리 결제가 그렇다

명령 요약 — 데이터가 갱신되면 이 네 줄

```
python pipeline/extract_load.py --source supabase
python pipeline/verify_load.py --source supabase
python qa/quality_check.py --source supabase
python bigquery/build.py --layer mart
```

① 새 행을 raw 로
② 빠진 행이 없는지
③ 믿어도 되는지 (430항목)
④ 마트를 다시 굽는다

stg 는 여기 없다. 뷰라서 ①이 끝나면 이미 최신이다 — SQL 을 고쳤을 때만 --layer stg 를 돌린다.

어디를 고치면 무엇이 바뀌나

고치는 곳	따라 바뀌는 것
bigquery/staging/*.sql	stg 뷰. --layer stg 를 돌리면 즉시 반영된다
bigquery/mart/*.sql	마트 표. 다시 구워야 반영된다
파일 이름의 번호	실행 순서. 앞 표를 읽는 표는 번호가 뒤여야 한다
새 지표가 필요할 때	임시 표를 만들지 말고 bigquery/mart/ 에 SQL 을 더한다 — 그래야 다음 사람도 같은 값을 본다